Ellucian ERP Integration Engine Enterprise Platform – Complete Technical Blueprint

This document is a complete end-to-end engineering and deployment guide for building the Ellucian ERP Integration Engine enterprise platform. It is designed so that a new engineer, architect, or technical lead can understand the system, deploy it, scale it, and maintain it.

1. Project Overview

The Integration Engine is a high-throughput API bridge that connects Ellucian Colleague ERP to multiple third-party systems across institutions. The system supports asynchronous processing, idempotency, retries, dead-letter queues, and cloud-native scaling. Primary goals: - 500k+ daily transactions - Sub-100ms API response time - Zero downtime deployments - Multi-tenant support - Enterprise reliability

---

2. Core Technology Stack

Backend: C# / .NET 6
Database: SQL Server / Azure SQL
Messaging: Azure Service Bus
ORM: Entity Framework Core
Testing: xUnit
Hosting: Azure App Service or AKS
Observability: OpenTelemetry + Application Insights
Infrastructure as Code: Terraform

---

3. High-Level Architecture

System Components: - API Gateway - Integration API - Azure Service Bus - Worker Services - SQL Server - ETL Pipelines - Observability Stack The system uses queue-based asynchronous processing to decouple services and ensure resilience.

---

4. Multi-Tenant Design

Every institution is treated as a tenant. All major tables include a TenantId. Configuration is tenant-specific: - API credentials - endpoint mappings - feature flags - retry policies

---

5. Database Schema

Core Tables: - Tenants - IntegrationRequests - IntegrationResponses - IntegrationJobs - AuditLogs - OutboxMessages Database design emphasizes: - append-only logging - indexed lookups - high-write throughput - idempotency enforcement

6. API Design

The API layer: - validates incoming requests - authenticates clients - checks idempotency keys - publishes messages to Azure Service Bus - returns asynchronous responses quickly Recommended endpoint structure: POST /api/integrations GET /api/status/{id}

7. Idempotency and Retry Layer

Idempotency ensures duplicate requests are not processed twice. Retry Strategy: 1st Retry: 5 seconds 2nd Retry: 30 seconds 3rd Retry: 2 minutes 4th Retry: 10 minutes 5th Retry: 30 minutes After max retries, messages move to a Dead Letter Queue (DLQ).

8. Azure Service Bus Architecture

Azure Service Bus acts as the backbone of asynchronous processing. Recommended setup: - Standard or Premium namespace - integration-queue - dead-letter queue - scheduled retries - topic subscriptions for scale

9. Worker Services

Worker services: - consume queue messages - transform payloads - route data to third-party systems - handle retries - update database records Workers should scale horizontally.

10. Adapter Pattern

Every third-party system should use its own adapter implementation. Benefits: - separation of concerns - easier testing - easier maintenance - simplified onboarding of new integrations

11. ETL Pipeline Strategy

ETL pipelines support: - nightly sync jobs - delta syncs - batch processing - near real-time replication Recommended approach: - Azure Functions - Worker Services - Change Tracking

12. Azure Deployment Guide

Azure Resources: - Resource Group - Azure SQL Database - Azure Service Bus - App Service Plan - API App Service - Worker App Service - Azure Key Vault Deployment Steps: 1. Create resource group 2. Create SQL Server and database 3. Configure firewall rules 4. Create Service Bus namespace and queue 5. Deploy API 6. Deploy Worker 7. Run EF Core migrations 8. Configure monitoring

13. Terraform Infrastructure

Terraform provisions: - resource groups - SQL - Service Bus - App Services - Key Vault Commands: terraform init terraform plan terraform apply

14. Kubernetes (AKS) Architecture

AKS is recommended for high-scale production deployments. Features: - autoscaling - rolling deployments - pod replication - advanced traffic routing - canary deployments

15. OpenTelemetry and Observability

Tracing stack: - OpenTelemetry - Jaeger - Application Insights Track: - API latency - queue depth - worker processing time - retry counts - external dependency failures

16. CI/CD Pipeline

GitHub Actions Pipeline: - restore packages - build - test - publish - deploy Recommended environments: - Development - Staging - Production

17. Security Strategy

Security Recommendations: - Managed Identities - Azure Key Vault - HTTPS everywhere - RBAC - API key rotation - secret scanning - tenant isolation

18. Load Testing

Load testing should simulate: - concurrent requests - retries - worker failures - queue bursts Recommended tool: - k6

19. Multi-Region Disaster Recovery

Production-grade failover includes: - Azure Front Door - SQL Failover Groups - Service Bus Geo-DR - Active/Passive regional architecture

20. Estimated Costs

Estimated Monthly Cost: Small Scale: $150–200/month Production Scale: $800–1500/month Primary costs: - App Service - SQL - Service Bus - Monitoring

---

21. Recommended Development Workflow

Recommended local setup: - Run API locally - Run Worker locally - Use Azure SQL - Use Azure Service Bus Avoid trying to simulate production scale on a local machine.

---

22. Scaling Strategy

Scaling Techniques: - horizontal pod autoscaling - queue partitioning - message batching - caching - async processing Future enhancements: - Redis - GraphQL gateway - event sourcing

---

23. Final Engineering Recommendations

This architecture is enterprise-grade and suitable for: - higher education systems - financial integrations - healthcare data pipelines The platform emphasizes: - resiliency - scalability - observability - maintainability

---

24. Example Azure CLI Commands

```
az group create --name integration-engine-rg --location eastus

az sql server create --name integration-sql-server --resource-group integration-engine-rg

az servicebus namespace create --name integration-bus --resource-group integration-engine-rg

az webapp create --resource-group integration-engine-rg --name integration-api-app
```

25. Conclusion

This document provides a complete engineering blueprint for building, deploying, scaling, and operating the Ellucian ERP Integration Engine enterprise platform. The architecture supports enterprise workloads, cloud-native deployments, and modern distributed systems best practices.